

JAVASCRIPT HANDWRITTEN NOTES

Part 2 - Page 1

1. PROMISE, ASYNC & AWAIT

• What is Callback ?

A callback is a function passed as an argument to another function and executed later.

```
function greet(name, callback) {  
  console.log("Hello " + name);  
  callback();  
}  
  
greet("Aman", function() {  
  console.log("Welcome!");  
});
```

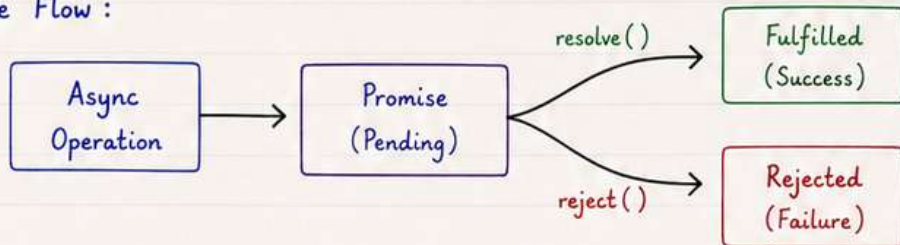
• What is Promise ?

A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation.

• Promise States :

- **Pending** - Initial state (neither fulfilled nor rejected)
- **Fulfilled** - Operation completed successfully
- **Rejected** - Operation failed

• Promise Flow :



• Promise Example :

```
let promise = new Promise(function(resolve, reject) {  
  let success = true;  
  if (success) {  
    resolve("Data Loaded Successfully");  
  } else {  
    reject("Something went wrong");  
  }  
});  
  
promise  
  .then(function(result) {  
    console.log(result);  
  })  
  .catch(function(error) {  
    console.log(error);  
  });
```

• Async / Await :

A cleaner way to work with Promises. It makes asynchronous code look synchronous and easy to read.

```
async function getData() {  
  try {  
    let data = await fetch("api/data");  
    let result = await data.json();  
    console.log(result);  
  } catch (error) {  
    console.log(error);  
  }  
}  
  
getData();
```

• Promise vs Async/Await

	Promise (.then/.catch)	Async / Await
Syntax	Chaining of .then() and .catch()	Cleaner and easier to read
Readability	Harder with multiple operations	Looks like synchronous code
Error Handling	Handled in .catch()	Handled using try...catch
Best For	Simple operations	Complex async logic

• Real World Example (API Call using Async/Await)

```
async function fetchUser() {  
  try {  
    let res = await fetch("https://jsonplaceholder.typicode.com/users/1");  
    let user = await res.json();  
    console.log(user);  
  } catch (err) {  
    console.log("Error:", err);  
  }  
}  
  
fetchUser();
```

JAVASCRIPT HANDWRITTEN NOTES

Part 2 - Page 2

2. MODERN JAVASCRIPT (ES6+)

- ES6 (ECMAScript 2015) introduced many powerful features that make code shorter, cleaner and easier to read.

1. Let vs const vs var

Keyword	Scope	Re-declare	Re-assign	Hoisting
var	Function Scoped	Yes	Yes	Yes
let	Block Scoped	No	Yes	Yes (TDZ)
const	Block Scoped	No	No	Yes (TDZ)

TDZ → Temporal Dead Zone

3. Arrow Functions

- Shorter syntax for writing functions.
- Does not have its own this.

```
// Normal function
const add = function(a, b) {
  return a + b;
}
// Arrow function
const add = (a, b) → a + b;
console.log(add(3, 4)); // 7
```

5. Destructuring

- Extract values from arrays or objects easily.

Array Destructuring

```
const numbers = [10, 20, 30];
const [a, b, c] = numbers;
console.log(a, b, c); // 10 20 30
```

Object Destructuring

```
const user = { name: "Aman", age: 25 };
const { name, age } = user;
console.log(name, age); // Aman 25
```

7. Rest Operator (...)

- Collects remaining elements into an array.

```
function sum(...numbers) {
  return numbers.reduce((total, num) → total + num, 0);
}
console.log(sum(1, 2, 3, 4, 5)); // 15

const [first, ...rest] = [10, 20, 30, 40];
console.log(first); // 10
console.log(rest); // [20, 30, 40]
```

2. Template Literals

- Use backticks (`) to create strings and embed expressions.

```
// Using template literal
let name = "Aman";
let age = 25;
console.log(`Hello ${name}, you are ${age} years old.`);

// Hello Aman, you are 25 years old.
```

4. Default Parameters

- Set default values for function parameters.

```
function greet(name = "Guest") {
  console.log(`Hello ${name}`);
}

greet(); // Hello Guest
greet("Aman"); // Hello Aman
```

6. Spread Operator (...)

- Expands an iterable (array, object, string, etc.).

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];
console.log(arr2); // [1, 2, 3, 4, 5]

const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 };
console.log(obj2); // { a: 1, b: 2, c: 3 }
```

Quick Summary

- ✓ let & const → Block scoped
- ✓ Template Literals → Use backticks and \${}
- ✓ Arrow Functions → Short & no own this
- ✓ Default Parameters → Set default values
- ✓ Destructuring → Extract values easily
- ✓ Spread Operator → Expand elements
- ✓ Rest Operator → Collect remaining elements

JAVASCRIPT HANDWRITTEN NOTES

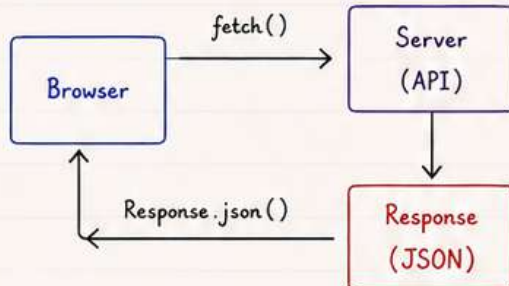
Part 2 - Page 3

3. FETCH API & JSON

• Fetch API

The Fetch API is used to make HTTP requests to the server.

Fetch Flow :



• Headers

```
const options = {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer your_token'
  }
};
fetch(url, options)
  .then(res => res.json())
  .then(data => console.log(data));
```

• Response.json()

Converts the response into JSON format.

```
fetch(url)
  .then(res => res.json()) // convert to JSON
  .then(data => console.log(data));
```

• JSON.stringify()

Converts a JavaScript object into a JSON string.

```
const user = { name: 'Aman', age: 25 };
const jsonString = JSON.stringify(user);
console.log(jsonString);
// {"name": "Aman", "age": 25 }
```

• GET Request

```
fetch('https://jsonplaceholder.typicode.com/users')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log("Error:", err));
```

• POST Request

```
fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    title: 'JavaScript Handbook',
    body: 'Learning Fetch API',
    userId: 1
  })
})
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log("Error:", err));
```

• JSON.parse()

Converts a JSON string into a JavaScript object.

```
const jsonString = '{"name": "Aman", "age": 25}';
const obj = JSON.parse(jsonString);
console.log(obj.name); // Aman
```

• Example (Complete Flow)

```
fetch('https://jsonplaceholder.typicode.com/users/1')
  .then(res => res.json())
  .then(user => {
    console.log("User:", user);
  })
  .catch(err => console.log("Error:", err));
```

Quick Notes

- ✓ fetch() returns a Promise.
- ✓ Always handle errors using .catch().
- ✓ Use headers for authentication or content type.
- ✓ Use JSON.parse() and JSON.stringify() to work with JSON data.

Common Status Codes

- 200 - OK (Success)
- 201 - Created (Success)
- 400 - Bad Request
- 401 - Unauthorized
- 404 - Not Found
- 500 - Internal Server Error

JAVASCRIPT HANDWRITTEN NOTES

Part 2 - Page 4

4. ARRAY METHODS (MOST USED)

- Array methods help us to perform operations on arrays easily and write clean code.

1. map()

- Creates a new array by applying a function to each element.

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(num → num * 2);
// [2, 4, 6, 8]
```

2. filter()

- Creates a new array with elements that satisfy the given condition.

```
const numbers = [1, 2, 3, 4, 5, 6];
const even = numbers.filter(num → num % 2 === 0);
// [2, 4, 6]
```

3. reduce()

- Reduces the array to a single value.
- It takes an accumulator and current value.

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((acc, num) → acc + num, 0);
// 10
```

4. find()

- Returns the first element that satisfies the condition.

```
const users = [
  {id: 1, name: 'Aman'},
  {id: 2, name: 'Rahul'}
];
const user = users.find(u → u.id === 2);
// { id: 2, name: 'Rahul' }
```

5. some()

- Returns true if at least one element satisfies the condition.

```
const numbers = [1, 3, 5, 8];
const hasEven = numbers.some(num → num % 2 === 0);
// true
```

6. every()

- Returns true if all elements satisfy the condition.

```
const numbers = [2, 4, 6, 8];
const allEven = numbers.every(num → num % 2 === 0);
// true
```

7. forEach()

- Executes a function for each element.
- Does not return a new array.

```
const numbers = [1, 2, 3];
numbers.forEach(num → {
  console.log(num);
});
// 1 2 3
```

8. sort()

- Sorts the elements of an array.
- By default, it converts elements to string and sorts in ascending order.

```
const fruits = ['banana', 'apple', 'orange'];
fruits.sort();
// ['apple', 'banana', 'orange']
```

map() vs forEach()

	map()	forEach()
Returns	New Array	Undefined
Purpose	Transform each element	Just perform operation
Use When	You need new modified array	You just need to loop

Quick Summary

- ✓ map() → Transform each element and return new array
- ✓ filter() → Filter elements based on condition
- ✓ reduce() → Reduce array to a single value
- ✓ find() → Find first matching element
- ✓ some() → Check if any element matches
- ✓ every() → Check if all elements match
- ✓ forEach() → Loop through elements (no return)
- ✓ sort() → Sort elements (default ascending)

JAVASCRIPT HANDWRITTEN NOTES

Part 2 - Page 5

5. CLOSURES, HOISTING & EXECUTION CONTEXT

1. HOISTING

- Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before code execution.

Example :

```
console.log(x);    // undefined
var x = 10;
console.log(x);    // 10
```

How it works internally :

Before Execution	After Hoisting
<pre>var x; // declaration console.log(x); x = 10; console.log(x);</pre>	<pre>var x; console.log(x); x = 10; console.log(x);</pre>

2. TEMPORAL DEAD ZONE (TDZ)

- TDZ is the time between the `let/const` declaration and its initialization.
- Accessing before initialization throws `ReferenceError`.

Example :

```
console.log(x);    // ReferenceError
let x = 10;
console.log(x);    // 10
```

Key Points :

- `var` → hoisted & initialized with `undefined`
- `let/const` → hoisted but not initialized (kept in TDZ)
- Cannot access `let/const` before initialization

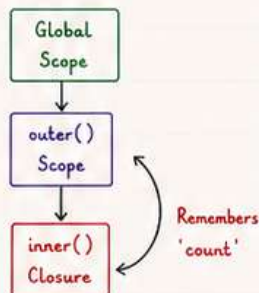
3. CLOSURES

- A closure is a function that remembers the variables from its outer (enclosing) function even after the outer function has finished execution.

Example :

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
  }
}
const fn = outer();
fn(); // 1
fn(); // 2
fn(); // 3
```

How it works ?



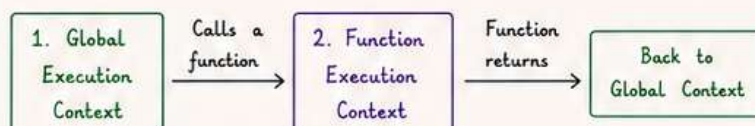
4. EXECUTION CONTEXT

- JavaScript code is executed inside an Execution Context.
- There are 3 types of Execution Contexts.

Types of Execution Contexts :

- Global Execution Context - Created first (at the start of the program)
- Function Execution Context - Created whenever a function is invoked
- Eval Execution Context - Created when 'eval()' is called

5. EXECUTION CONTEXT - HOW IT WORKS ?



Inside Each Execution Context
(Two Phases)

1. Creation Phase
 - Memory is created
 - Variables are stored with `undefined`
2. Execution Phase
 - Code is executed line by line
 - Values are assigned and functions run

Quick Comparison

	var	let	const
Hoisting	Yes (initialized with undefined)	Yes (but in TDZ)	Yes (but in TDZ)
Re-declare in same scope	Yes	No	No
Re-assign value	Yes	Yes	No

Key Takeaways

- ✓ Hoisting helps JS move declarations to the top.
- ✓ `let/const` are hoisted but kept in TDZ.
- ✓ Closures allow functions to remember outer variables.
- ✓ Execution Context is the core of how JS code runs.

JAVASCRIPT HANDWRITTEN NOTES

Part 2 - Page 6

6. THIS KEYWORD, CALL(), APPLY() & BIND()

1. this Keyword

- this refers to the object that is executing the current function.
- Its value depends on how a function is called.

Where it is used	What this refers to
In Global scope	window object
In Object method	The object
In Constructor	Newly created object
In Event handler	The element that triggered the event
In Arrow function	Inherited from surrounding scope

2. call() Method

- Calls a function with a given this value and arguments provided one by one.

```
const user = {
  name: 'Aman',
  greet: function (msg) {
    console.log(msg + ', ' + this.name);
  }
};
user.greet('Hello'); // Hello, Aman
user.greet.call({ name: 'Rahul' }, 'Hi'); // Hi, Rahul
```

3. apply() Method

- Similar to call(), but arguments are passed as an array.

```
user.greet.apply({ name: 'Rahul' }, ['Hey']);
// Hey, Rahul
```

4. bind() Method

- Returns a new function with a fixed this value.
- Does not call the function immediately.

```
const newGreet = user.greet.bind({
  name: 'Rahul' }, 'Namaste');
newGreet(); // Namaste, Rahul
```

5. Quick Difference

Method	Calls Immediately?	Arguments Passed As	Returns New Function?
call()	Yes	List of values	No
apply()	Yes	Array	No
bind()	No	List of values	Yes

Examples

1. this in Object Method

```
const obj = {
  name: 'Aman',
  show: function () {
    console.log(this.name);
  }
};
obj.show(); // Aman
```

2. this in Constructor

```
function Person(name) {
  this.name = name;
}
const p1 = new Person('Aman');
console.log(p1.name); // Aman
```

3. this in Event Handler (Browser)

```
<button id="btn">Click Me</button>
<script>
  document.getElementById('btn').onclick = function(){
    console.log(this.innerText);
  };
</script>
// Output: Click Me
```

Key Takeaways

- ✓ this depends on how a function is called.
- ✓ call() and apply() call the function immediately.
- ✓ bind() returns a new function.
- ✓ Use call(), apply(), bind() to borrow methods from other objects.

Real World Use

- Borrow methods from arrays/objects.
- Set a fixed context for a function.
- Useful in libraries and frameworks (React, jQuery, etc.).

JAVASCRIPT HANDWRITTEN NOTES

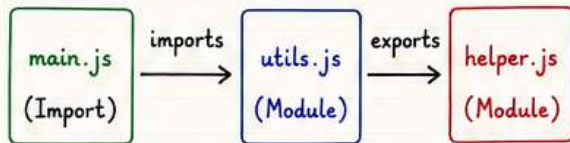
Part 2 - Page 7

7. MODULES & ERROR HANDLING

1. Modules in JavaScript

- Modules help to split code into separate files.
- Improves reusability and maintainability.

How it works ?



2. Exporting

- Named Export

```
// math.js
export const add = (a, b) → a + b;
export const sub = (a, b) → a - b;
```

- Default Export

```
// user.js
const user = { name: 'Aman', age: 25 };
export default user;
```

3. Importing

- Named Import

```
// main.js
import { add, sub } from './math.js';
console.log(add(2, 3)); // 5
```

- Default Import

```
// app.js
import user from './user.js';
console.log(user.name); // Aman
```

- Import Everything

```
// all.js
import * as math from './math.js';
console.log(math.add(4, 6)); // 10
```

Quick Summary

- ✓ Use export to share code from a module.
- ✓ Use import to use code in another file.
- ✓ Named export → use { } while importing.
- ✓ Default export → no { } needed.
- ✓ Modules make code clean and scalable.

4. try...catch...finally

- Handles runtime errors gracefully.

```
try {
  let result = 10 / 0;
  console.log(result);
} catch (error) {
  console.log('Error:', error.message);
} finally {
  console.log('This will always run');
```

5. throw Statement

- Used to create custom errors.

```
function validateAge(age) {
  if (age < 18) {
    throw new Error('Age must be 18 or above!');
  }
  return 'Valid Age';
}

console.log(validateAge(16)); // Error
```

6. Example (Complete Flow)

```
function divide(a, b) {
  if (b === 0) {
    throw new Error('Cannot divide by zero');
  }
  return a / b;
}

try {
  let res = divide(10, 0);
  console.log('Result:', res);
} catch (err) {
  console.log('Caught Error →', err.message);
} finally {
  console.log('Execution Completed');
```

Common Errors

- ReferenceError → Accessing undeclared variable
- TypeError → Wrong type of value/method
- SyntaxError → Mistake in code syntax
- RangeError → Number out of valid range
- Custom Error → Created using throw new Error()

JAVASCRIPT HANDWRITTEN NOTES

Part 2 - Page 8

8. BROWSER STORAGE & TIMERS

1. Web Storage

- Used to store data in the browser.

Feature	localStorage	sessionStorage
Scope	Permanent	Until tab is closed
Data Retention	No expiration	Cleared on tab close
Capacity	~5MB	~5MB
Access	All tabs/windows	Same tab only

localStorage Example

```
// Set Item
localStorage.setItem('name', 'Aman');

// Get Item
let name = localStorage.getItem('name');
console.log(name); // Aman

// Remove Item
localStorage.removeItem('name');

// Clear All
localStorage.clear();
```

sessionStorage Example

```
sessionStorage.setItem('token', 'abc123');
let token = sessionStorage.getItem('token');
sessionStorage.removeItem('token');
sessionStorage.clear();
```

4. Clearing Timers

A. clearTimeout()

- Stops the setTimeout from executing.

```
let timeoutId = setTimeout(() => {
  console.log('You will not see this');
}, 3000);

clearTimeout(timeoutId);
```

Quick Summary

- ✓ localStorage → Stores data with no expiration.
- ✓ sessionStorage → Stores data until tab is closed.
- ✓ Cookies → Small data sent with every request.
- ✓ setTimeout → Run code once after delay.
- ✓ setInterval → Run code repeatedly at interval.
- ✓ clearTimeout / clearInterval → Stop timers.

2. Cookies (Basic)

- Cookies are small pieces of data stored by the browser.
- They are sent with every HTTP request.

```
// Create Cookie
document.cookie = "user=Aman; path=/; max-age=3600";

// Read Cookie
console.log(document.cookie); // user=Aman

// Delete Cookie
document.cookie = "user=; path=/; max-age=0";
```

3. Timers

- Used to run code after a delay or at intervals.

A. setTimeout()

- Executes code once after a delay.

```
setTimeout(() => {
  console.log('Hello after 2 seconds');
}, 2000); // 2000ms = 2s
```

B. setInterval()

- Executes code repeatedly at a fixed interval.

```
let id = setInterval(() => {
  console.log('This runs every 3 seconds');
}, 3000); // 3000ms = 3s
```

B. clearInterval()

- Stops the setInterval from running.

```
let count = 0;
let intervalId = setInterval(() => {
  count++;
  console.log(count);
  if (count === 5) {
    clearInterval(intervalId); // Stop after 5 times
  }
}, 1000);
```

Common Use Cases

- localStorage → Save user preferences, theme, login info.
- sessionStorage → Store temporary data, form info.
- Cookies → Manage sessions, authentication.
- Timers → Sliders, auto-save, countdowns, animations.

 **Tip:** Always clear intervals when not needed to avoid performance issues.



COMMENT PDF

— TO GET —

PDF NOTES



Aman Views

Java • SQL • System Design • DSA
Backend Development • Web Development
QA Testing

Handwritten Notes



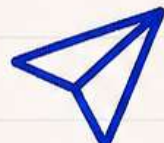
TO GET LINK OF ALL OUR NOTES,
CHECK THE LINK IN BIO.



SAVE



FOLLOW



SHARE

— Thank you for your support! ❤️